DISCIPLINA · 60 HORAS

Modelo Físico e SQL

Módulo 03 — DDL, DML e consultas no MariaDB

Semanas 5 a 12 · Eixo Informação e Comunicação

ONDE ESTAMOS

A disciplina em 3 módulos

CONCLUÍDO



MÓDULO 01

Visão Geral

Banco de dados, SGBD e introdução à modelagem.

Semana 1

CONCLUÍDO



MÓDULO 02

Modelagem de Dados

Modelo conceitual e lógico: entidades, atributos e relacionamentos.

Semanas 2 a 4

VOCÊ ESTÁ AQUI



MÓDULO 03

Modelo Físico e SQL

Criar o banco de verdade e manipular os dados com SQL.

Semanas 5 a 12

PLANO DO MÓDULO

O caminho até o banco funcionando

Etapa 1 · Do lógico ao físico	O que muda quando escolhemos o SGBD.
Etapa 2 · A linguagem SQL	Os grupos de comandos: DDL, DML, DQL, DCL e DTL.
Etapa 3 · DDL — criar a estrutura	Banco, tabelas, tipos de dados e restrições.
Etapa 4 · DML — manipular os dados	INSERT, UPDATE e DELETE.
Etapa 5 · Consultas — SELECT	Filtrar, ordenar, agrupar e juntar tabelas.
Etapa 6 · Projeto final	Do modelo conceitual ao banco em funcionamento.

MÓDULO 03 · ETAPA 1

Do lógico ao físico

01

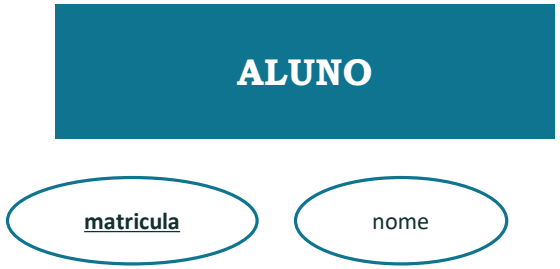
O que muda quando escolhemos o SGBD

OS TRÊS NÍVEIS

Relembrando onde estamos

1 · CONCEITUAL

A ideia, sem tecnologia



Entidades, atributos e relacionamentos.

2 · LÓGICO

Vira esquema de tabela

```
ALUNO (matricula, nome)
```

Tabelas, colunas e chaves — ainda sem SGBD.

3 · FÍSICO

Vira comando SQL

```
CREATE TABLE aluno (
  matricula INT
    PRIMARY KEY,
  nome VARCHAR(100)
);
```

Tipos de dados e comandos no MariaDB.

É a MESMA informação em três formas. Agora chegamos à terceira.

FERRAMENTAS

Onde vamos trabalhar



MariaDB / MySQL

O SGBD que guarda e gerencia os dados. É quem executa os comandos e onde o banco existe de verdade.



MySQL Workbench

A interface visual onde digitamos o SQL, executamos e vemos o resultado em forma de tabela.

Todo comando que veremos é digitado no Workbench e executado pelo MariaDB.

MÓDULO 03 · ETAPA 2

A linguagem SQL

02

Os grupos de comandos

SQL

O que é e para que serve

Definição

SQL (Structured Query Language) é a linguagem usada para conversar com o banco de dados relacional: criar estruturas, inserir, alterar e consultar dados.

É um padrão: o que você aprender aqui serve para PostgreSQL, Oracle e SQL Server, com pequenas diferenças.

UM COMANDO SIMPLES

```
SELECT nome
FROM aluno;
```

Lê-se: "traga a coluna nome da tabela aluno".

Todo comando SQL termina com ponto e vírgula.

SQL

Os cinco grupos de comandos



DDL

Definição

Cria e altera a estrutura.

`CREATE` · `ALTER` · `DROP`



DML

Manipulação

Mexe nos dados guardados.

`INSERT` · `UPDATE` · `DELETE`



DQL

Consulta

Busca e devolve dados.

`SELECT`



DCL

Controle

Permissões de acesso.

`GRANT` · `REVOKE`



DTL

Transação

Confirma ou desfaz alterações.

`COMMIT` · `ROLLBACK`

Os três primeiros são o foco do módulo. DCL e DTL veremos de forma introdutória.

MÓDULO 03 · ETAPA 3

DDL

03

Criando a estrutura do banco

DDL

Criando e selecionando o banco

Antes de criar qualquer tabela, é preciso criar o banco e dizer ao SGBD que vamos trabalhar nele.

```
CREATE DATABASE escola;

USE escola;

SHOW DATABASES;

DROP DATABASE escola;
```

CREATE DATABASE

cria o banco.

USE

seleciona o banco.

SHOW DATABASES

lista os bancos

apaga tudo.



Atenção: DROP DATABASE apaga o banco inteiro, com todos os dados, e não existe desfazer. Confira o nome antes de executar.

DDL

Tipos de dados mais usados

Cada coluna precisa de um tipo. É aqui que o modelo lógico ganha detalhe técnico.

`INT`	Números inteiros	*ex.: idade, quantidade*
`VARCHAR(n)`	Texto de tamanho variável, até n caracteres	*ex.: nome, email*
`CHAR(n)`	Texto de tamanho fixo	*ex.: UF, sigla*
`DATE`	Data (AAAA-MM-DD)	*ex.: nascimento, matrícula*
`DECIMAL(m,d)`	Número com casas decimais	*ex.: preço, nota*
`BOOLEAN`	Verdadeiro ou falso	*ex.: ativo, pago*

DECIMAL(10,2) guarda até 10 dígitos, sendo 2 depois da vírgula. VARCHAR(100) aceita até 100 caracteres.

DDL

Anatomia do CREATE TABLE

```
CREATE TABLE aluno (
  matricula INT PRIMARY KEY,
  nome VARCHAR(100) NOT NULL,
  nascimento DATE,
  email VARCHAR(80) UNIQUE,
  ativo BOOLEAN DEFAULT TRUE
);
```

Lendo linha a linha

- nome da tabela, no singular;
- uma linha por coluna, separadas por vírgula;
- cada coluna: nome + tipo + restrições;
- fecha parênteses e ponto e vírgula.

Do lógico para o físico: ALUNO (matricula, nome, nascimento, email, ativo) virou exatamente este comando.

DDL

Restrições (constraints)

As restrições são as regras que o SGBD passa a cobrar sozinho. É assim que ele garante a integridade dos dados.

PRIMARY KEY	Identifica cada linha. Não repete e não fica vazia.	`matricula INT PRIMARY KEY`
NOT NULL	A coluna não pode ficar vazia.	`nome VARCHAR(100) NOT NULL`
UNIQUE	O valor não pode se repetir na tabela.	`email VARCHAR(80) UNIQUE`
DEFAULT	Valor assumido quando nada é informado.	`ativo BOOLEAN DEFAULT TRUE`
AUTO_INCREMENT	O SGBD gera o número sozinho, de 1 em 1.	`id INT AUTO_INCREMENT`
FOREIGN KEY	Liga a coluna à chave primária de outra tabela.	`FOREIGN KEY (turma) REFERENCES turma(codigo)`

DDL

Chave estrangeira: ligando as tabelas

A FK é o que faz o relacionamento do diagrama existir de verdade dentro do banco.

```
CREATE TABLE turma (
  codigo INT PRIMARY KEY,
  serie VARCHAR(20)
);

CREATE TABLE aluno (
  matricula INT PRIMARY KEY,
  nome VARCHAR(100),
  turma INT,
  FOREIGN KEY (turma) REFERENCES turma(codigo)
);
```



Regra de ouro

A tabela referenciada precisa existir ANTES. Por isso criamos turma antes de aluno.

Depois disso, o SGBD recusa um aluno com turma inexistente.

Essa recusa é a integridade referencial trabalhando por você.

DDL · EXEMPLO COMPLETO

O banco da escola, do zero

```
CREATE DATABASE escola;
USE escola;

CREATE TABLE turma (
  codigo INT PRIMARY KEY,
  serie VARCHAR(20) NOT NULL
);

CREATE TABLE aluno (
  matricula INT PRIMARY KEY,
  nome VARCHAR(100) NOT NULL,
  turma INT,
  FOREIGN KEY (turma)
    REFERENCES turma(codigo)
);
```

```
CREATE TABLE disciplina (
  codigo INT PRIMARY KEY,
  nome VARCHAR(60)
);

CREATE TABLE cursa (
  aluno INT,
  disciplina INT,
  nota DECIMAL(4,2),
  PRIMARY KEY (aluno, disciplina),
  FOREIGN KEY (aluno)
    REFERENCES aluno(matricula),
  FOREIGN KEY (disciplina)
    REFERENCES disciplina(codigo)
);
```

A tabela CURSA é a associativa do N:N: duas FKs que, juntas, formam a PK composta — e guarda a nota, atributo do relacionamento.

DDL

ALTER TABLE: mudando o que já existe

Nem sempre acertamos de primeira. O ALTER permite corrigir a estrutura sem recriar a tabela.

Adicionar coluna

```
ALTER TABLE aluno ADD telefone VARCHAR(15);
```

Mudar o tipo

```
ALTER TABLE aluno MODIFY nome VARCHAR(150);
```

Renomear coluna

```
ALTER TABLE aluno CHANGE email email_aluno VARCHAR(80);
```

Remover coluna

```
ALTER TABLE aluno DROP COLUMN telefone;
```

DDL · ATENÇÃO

DROP, TRUNCATE e DELETE: não confunda

`DROP TABLE`

Apaga a tabela inteira: estrutura e dados. A tabela deixa de existir.

`TRUNCATE TABLE`

Apaga todos os dados, mas mantém a tabela vazia e pronta para uso.

`DELETE FROM`

Apaga linhas específicas, conforme o WHERE. É um comando DML.



Regra de sala: antes de executar DROP ou DELETE, leia o comando em voz alta e confirme o nome da tabela e o WHERE.

MÓDULO 03 · ETAPA 4

DML

04

Inserindo, alterando e apagando dados

DML

INSERT: colocando dados na tabela

```
INSERT INTO turma (codigo, serie)
VALUES (1, '1 ano A');

INSERT INTO aluno (matricula, nome, turma)
VALUES (101, 'Ana Lima', 1);
```

A estrutura

```
INSERT INTO tabela (colunas) VALUES
(valores);
```

Inserindo vários de uma vez:

```
INSERT INTO aluno (matricula, nome, turma) VALUES
  (102, 'Bruno Sa', 1),
  (103, 'Carla Reis', 2);
```

Texto e data vão entre aspas simples; números, não.

DML

A ordem de inserção importa

Com chave estrangeira, o SGBD exige que o dado referenciado já exista. Isso define a ordem dos INSERTs.

CERTO

```
INSERT INTO turma ...
  (cria a turma 1)

INSERT INTO aluno ...
  (aluno da turma 1)
```

A turma já existe quando o aluno chega.

ERRADO

```
INSERT INTO aluno ...
  (turma 9 nao existe)

ERRO 1452: Cannot add
or update a child row
```

O SGBD recusa e protege a integridade.

DML

UPDATE: alterando dados existentes

```
UPDATE aluno
SET nome = 'Ana Lima Souza'
WHERE matricula = 101;
```

A estrutura

```
UPDATE tabela SET coluna = valor WHERE condicao;
```

Alterando mais de uma coluna:

```
UPDATE aluno SET nome = 'Bruno Sa Filho', turma = 2 WHERE matricula = 102;
```



Nunca esqueça o WHERE: sem ele, o UPDATE altera TODAS as linhas da tabela. É o erro mais caro que um iniciante comete.

DML

DELETE: apagando linhas

```
DELETE FROM aluno
WHERE matricula = 103;
```

A estrutura

```
DELETE FROM tabela WHERE condicao;
```

Antes de apagar, teste a condição com um SELECT:

```
SELECT * FROM aluno WHERE matricula = 103;   -- confere o que sera apagado
DELETE  FROM aluno WHERE matricula = 103;    -- so depois apaga
```



Hábito que salva: rode sempre o SELECT com a mesma condição antes do DELETE ou do UPDATE. Se o SELECT trouxe o que você esperava, pode executar.

MÓDULO 03 · ETAPA 5

Consultas

05

Buscando a informação com SELECT

SELECT

A consulta básica

```
SELECT * FROM aluno;

SELECT nome, turma FROM aluno;

SELECT nome AS aluno FROM aluno;
```

Lendo

- * traz todas as colunas;
- ou liste só as que interessam;
- AS renomeia a coluna no resultado.

O resultado vem em forma de tabela:

matricula	nome	turma
101	Ana Lima	1
102	Bruno Sa	1
103	Carla Reis	2

SELECT

WHERE: filtrando os resultados

O WHERE é a peneira: só passam as linhas que atendem à condição.

=	igual a	`WHERE turma = 1`
<> ou !=	diferente de	`WHERE turma <> 1`
> < >= <=	maior / menor	`WHERE nota >= 7`
BETWEEN	entre dois valores	`WHERE nota BETWEEN 7 AND 10`
IN	está na lista	`WHERE turma IN (1, 2, 3)`
LIKE	parecido com	`WHERE nome LIKE 'Ana%'`
AND / OR	combina condições	`WHERE turma = 1 AND nota >= 7`

No LIKE, o % substitui qualquer sequência: 'Ana%' acha Ana, Ana Lima, Anabela.

SELECT

Ordenando e limitando



ORDER BY

Ordena o resultado. ASC é crescente (padrão) e DESC é decrescente.

```
SELECT nome, nota FROM cursa
ORDER BY nota DESC;
```



LIMIT

Mostra apenas as primeiras linhas do resultado.

```
SELECT nome FROM aluno
LIMIT 5;
```



DISTINCT

Remove valores repetidos do resultado.

```
SELECT DISTINCT turma
FROM aluno;
```

Dá para combinar tudo: SELECT ... WHERE ... ORDER BY ... LIMIT ...

SELECT

Funções de agregação e GROUP BY

Em vez de listar linha a linha, estas funções resumem os dados em um número.

COUNT()	quantas linhas	`COUNT(*)`
SUM()	soma os valores	`SUM(nota)`
AVG()	calcula a média	`AVG(nota)`
MAX()	maior valor	`MAX(nota)`
MIN()	menor valor	`MIN(nota)`

Com GROUP BY

Agrupa por uma coluna e aplica a função em cada grupo:

```
SELECT turma,
  COUNT(*) AS total,
  AVG(nota) AS media
FROM cursa
GROUP BY turma;
```

Resposta: "quantos alunos e qual a média de cada turma".

SELECT

JOIN: consultando duas tabelas juntas

A FK liga as tabelas na estrutura; o JOIN usa essa ligação na hora da consulta.

```
SELECT aluno.nome, turma.serie
FROM aluno
INNER JOIN turma
  ON aluno.turma = turma.codigo;
```

A ideia

O ON diz qual coluna de uma tabela corresponde à da outra — normalmente FK = PK.

Resultado: cada aluno já com o nome da sua turma, e não apenas o código.

nome	serie
Ana Lima	1 ano A
Bruno Sa	1 ano A
Carla Reis	1 ano B

MÓDULO 03 · ETAPA 6

DCL e DTL

06

Permissões e transações

DCL E DTL

Os dois grupos que faltavam

DCL — Controle de acesso

Define quem pode fazer o quê dentro do banco.

```
GRANT SELECT ON escola.aluno
  TO 'professor'@'localhost';

REVOKE SELECT ON escola.aluno
  FROM 'professor'@'localhost';
```

DTL — Transações

Permite confirmar ou desfazer um conjunto de alterações.

```
START TRANSACTION;
UPDATE aluno SET turma = 2
  WHERE matricula = 101;
ROLLBACK;    -- desfaz
-- ou COMMIT;  -- confirma
```

O ROLLBACK é a rede de segurança: se o comando saiu errado, dá para voltar atrás antes do COMMIT.

ROTEIRO PRÁTICO

A ordem de trabalho no Workbench

1	**Criar o banco**	CREATE DATABASE e USE
2	**Criar as tabelas**	CREATE TABLE, das independentes para as que têm FK
3	**Inserir os dados**	INSERT, respeitando a ordem das FKs
4	**Conferir**	SELECT * em cada tabela
5	**Consultar**	SELECT com WHERE, ORDER BY, JOIN e agregações
6	**Ajustar quando preciso**	UPDATE e DELETE, sempre com WHERE

Salve sempre o script .sql: é ele que permite recriar o banco inteiro do zero.

ATENÇÃO

Erros mais comuns em SQL



Esquecer o ponto e vírgula

O comando não executa e o Workbench fica esperando.



UPDATE ou DELETE sem WHERE

Altera ou apaga a tabela inteira. Teste antes com SELECT.



Inserir filho antes do pai

Erro de FK: a turma precisa existir antes do aluno.



Esquecer as aspas no texto

Texto e data vão entre aspas simples; números, não.

RECAPITULANDO

O que você já sabe fazer

- Traduzir o modelo lógico em comandos SQL de criação.
- Reconhecer os grupos DDL, DML, DQL, DCL e DTL.
- Criar banco e tabelas com tipos de dados e restrições.
- Inserir, alterar e apagar dados com segurança.
- Consultar com SELECT, WHERE, ORDER BY, GROUP BY e JOIN.
- Trabalhar no MySQL Workbench seguindo um roteiro.

Próximo passo — o projeto final: do modelo conceitual ao banco funcionando.